

编译和使用说明

本项目编译和使用都使用make管理

环境

开发环境

```
OS: Arch Linux x86_64
Host: HP Pavilion Aero Lapt-getop 13-belxxx
Kernel: 6.11.9-arch1-1
DE: Plasma 6.2.3
WM: KWin
Theme: Breeze-Dark [GTK2], Breeze [GTK3]
Icons: Colloid [GTK2/3]
Terminal: konsole
```

移植测试环境

测试环境1

```
OS: Kali GNU/Linux Rolling x86_64
Host: 911Air Type1Version
Kernel: 6.11.2-amd64
DE: GNOME 45.3
WM: Mutter
Theme: adw-gtk3-dark [GTK2/3]
Icons: Flat-Remix-Blue-Dark [GTK2/3]
Terminal: gnome-terminal
```

测试环境2

```
OS: Ubuntu 22.04.3 LTS on Windows 10 x86_64
Kernel: 5.15.167.4-microsoft-standard-WSL2
Theme: Yaru [GTK3]
Icons: Yaru [GTK3]
Terminal: vscode
```

依赖项安装(以Ubuntu22.04为例)

项目的图形化部分使用QT5开发，故需要QT5的有关依赖

```
sudo apt-get update
sudo apt-get install qtbase5-dev qtchooser qt5-qmake qttools5-dev-tools
```

C++编译工具链安装：

```
sudo apt-get update
sudo apt-get install build-essential cmake
```

显示有关依赖安装：

```
sudo apt-get update
sudo apt-get install libglu1-mesa-dev freeglut3-dev mesa-common-dev
```

编译

打开项目根目录，在终端中运行：

```
make all
```

请注意：项目根目录所在路径中不能含有空格、括号、中文字符

运行

在项目根目录运行：

```
make run
```

请再次注意：项目根目录所在路径中不能含有空格、括号、中文字符

使用说明

主界面



界面主要分为4个区域，分别为：

- 文件管理区
- 缓存区
- 词法分析器
- 语法分析器

下面将展示每个区域的作用。

1. 文件管理区

文件管理区提供了六个按钮：

打开源程序文件

- 点击按钮，将会打开一个文件资源管理器，使用者可以选择对应的源程序文件，文件中的内容会被载入到源程序区供使用者查看和修改。

保存

- 点击按钮，将会直接将编辑框中文件保存到源文件中，若为全新编写，则会打开一个文件资源管理器，使用者可以创建文件，将源程序区的内容保存到目标文件中。

另存为

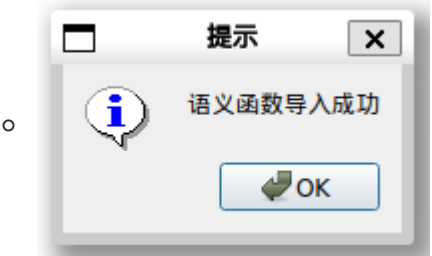
- 点击按钮，将会打开一个文件资源管理器，使用者可以选择文件，将源程序区的内容保存到被选中的文件里。

编辑语义函数

- 点击按钮，将会弹出新对话框，在新对话框中可以编辑语义函数。



- 点击新对话框的保存按钮，将会直接将编辑框中文件保存到源语义函数文件中，并将语义函数进行更新。
- 点击新对话框的另存为按钮，将会打开一个文件资源管理器，使用者可以选择文件，将编辑框中的内容保存到被选中的文件里。
- 点击退出按钮，则会直接退出。
- 若语义函数更新成功，会弹出窗口提示成功，否则会弹出窗口给出编译命令，届时可以将命令在终端输入获取具体编译失败信息。



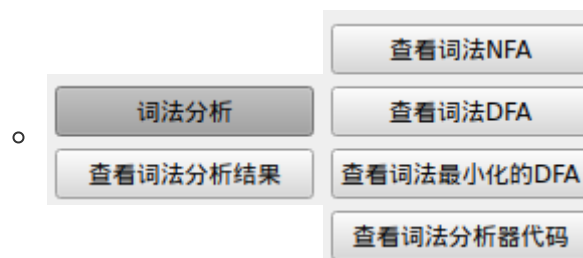
编辑语法/词法文件

- 点击按钮，将会弹出新对话框，在新对话框中可以编辑语法/词法文件。
 - 点击新对话框的保存按钮，将会直接将编辑框中文件保存到源语法/词法文件中，不会触发重新导入，需要退出编辑后手动导入。
 - 点击新对话框的另存为按钮，将会打开一个文件资源管理器，使用者可以选择文件，将编辑框中的内容保存到被选中的文件里。
 - 点击退出按钮，则会直接退出。

2. 缓存区

缓存区保存了最近一次词法正则、语法BNF文件和语义函数的输入信息。

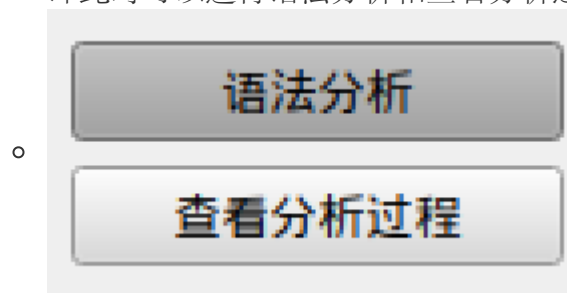
- 词法来源如果不为空，那么随时可以查看词法分析器有关信息以及进行词法分析。
 - 即可以点击词法分析器有关的所有按钮。



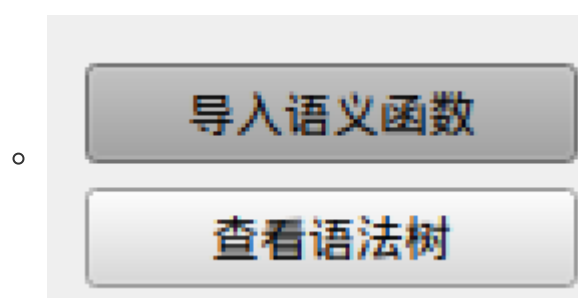
- 语法来源如果不为空，可以随时查看LR(1)DFA等语法分析器相关信息。
 - 即可以点击语法分析器语法相关的按钮。



- 词法来源和语法来源同时不为空时，可以进行语法分析并查看语法分析的有关信息。
 - 即此时可以进行语法分析和查看分析过程。



- 词法来源、语法来源、语义函数来源同时不为空时，可以查看语法树。



3. 词法分析器

词法分析器由词法分析器生成器生成。

词法分析器生成器的输入

- 词法分析器读入.rul后缀的正则文件。
 - .rul后缀的正则文件规则如下：
 - >>符号标识词法正则文件内的注释，当以该符号开始时，此行会被忽略。

- Space、Tab和Enter会被自动识别。
- 若存在注释，长注释的开闭符号、单行注释的开始符号必须于文件头部声明：
 - 长注释开：`_open_comment =`
 - 长注释闭：`_close_comment =`
 - 单行注释开：`_oneLine_comment_open =`
- 注释符号的书写应参考下例：

```
_open_comment = /*
_close_comment = */
_oneLine_comment_open = //
```

- 存在预定义的正则：letter、digit。
 - letter定义为：`[a-zA-Z]`
 - digit定义为：`[0-9]`
- 如需使用预定义的正则，可参考下例：

```
id 10 = (_|letter)(_|letter|digit)*
```

- 需要自行定义的正则应书写为：

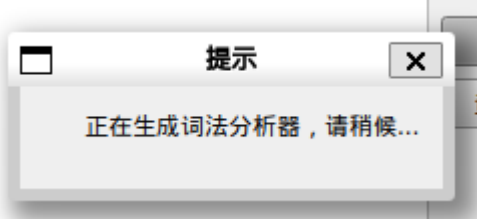
```
<正则类型(可无意义)> <起始编码> = <第一个待编码正则> <第二个待编码正则> <第三个待编码正则> ....
```

- 自定义正则的书写示例如下：

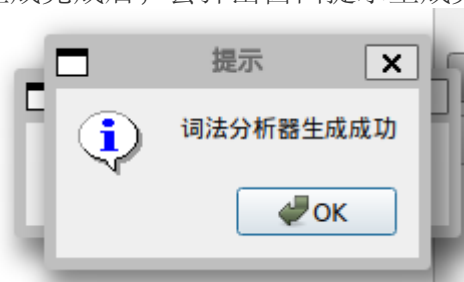
```
keywords 1000 = if then else while do return void int
float double
```

载入词法正则文件

- 点击按钮“载入词法正则文件”，会打开一个文件资源管理器，用户选择对应的正则文件即可进行导入。
- 由于将正则转为对应代码的时间开销比较大，请用户耐心等待。



- 生成完成后，会弹出窗口提示生成完成。



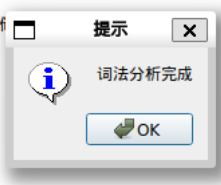
- 若成功导入，缓存区的“当前词法来源”会更新为所选文件路径。

词法分析

- 当源程序编辑框不为空时，点击“词法分析”按钮，待词法分析结束时有弹窗提示词法分析完成。

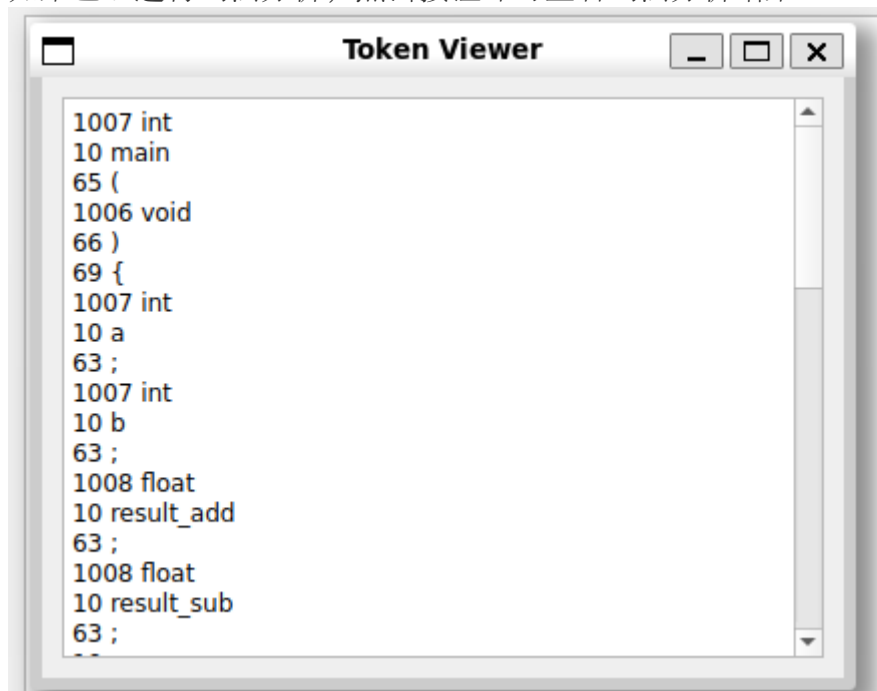
// miniC示例程序：计算两个数的和与差

```
int main(void) {  
    int a;  
    int b; // 定义两个整型变量  
    float result_add;  
    float result_sub; // 定义两个浮点型变量用于存储  
  
    // 赋值操作  
    a = 10; // 第一个数  
    b = 20; // 第二个数  
  
    // 加法和减法  
    result_add = a + b; // 计算和  
    result_sub = a - b; // 计算差  
  
    // 输出结果  
    print(result_add);  
  
    print(result_sub);  
  
    return 0; // 程序正常结束  
}
```



查看词法分析结果

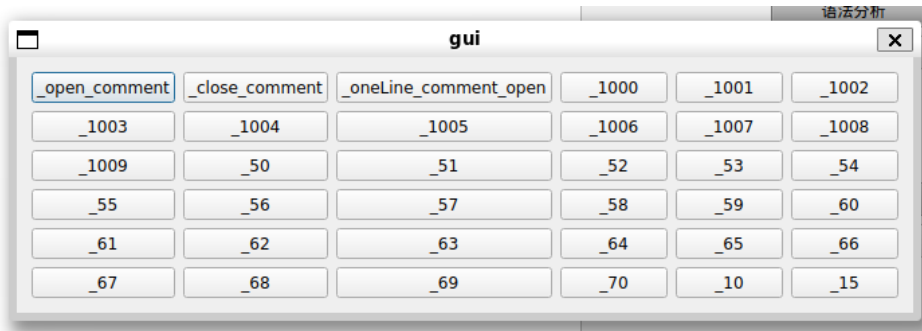
- 如果已经进行词法分析，点击按钮即可查看词法分析结果。



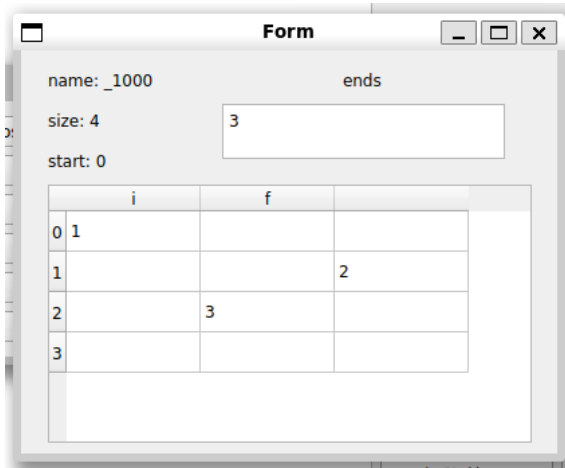
- 除此之外，也可以在<项目根目录>/build/token.lex读取到上述内容。

查看词法NFA

- 如果词法分析器生成完成，点击此按钮可以分别展示当前词法分析器所能识别的所有正则表达式的NFA。



- 点击其中一个即可查看具体结果。



-

查看词法DFA

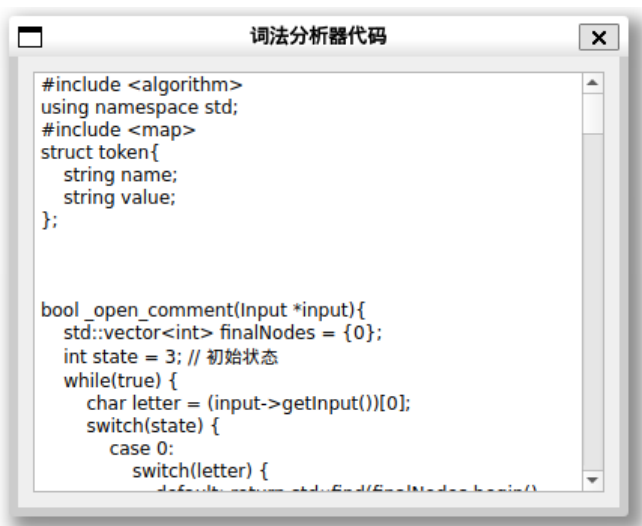
- 基本操作逻辑同上。

查看词法最小化DFA

- 基本操作逻辑同上。

查看词法分析器代码

- 点击按钮，即可查看词法分析器的源代码code.cpp和getInput.h。



-

4. 语法分析器

语法分析器需要导入BNF语法文件。

BNF语法文件的格式

- 语法分析器读入.grm后缀的语法文件。
 - .grm后缀的正则文件规则如下：
 - == token ==后为终结符的编码规则。
 - 终结符编码规则类似词法正则的编码规则，但去掉了<正则类型(可无意义)>和=，编码规则为：<开始编码> <终结符1> <终结符2>.....可以参考下面示例：

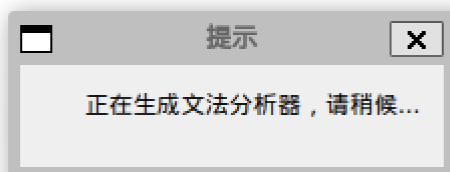
```
1000 if then else while do return void int float double
```

- == grammar ==后为文法规则。
- 每条文法都有推导符号->，推导符号左侧为开始符号，推导符号右侧为推导式，每个符号以空格间隔，简单示例如下：

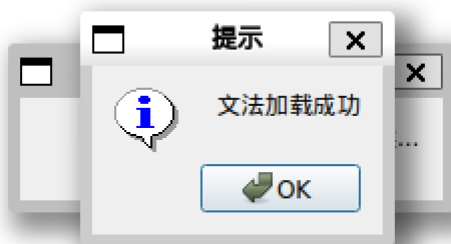
```
definition -> variable-definition | function-definition
```

载入BNF语法文件

- 点击按钮“载入BNF语法文件”，会打开一个文件资源管理器，用户选择对应的BNF语法文件即可进行导入。

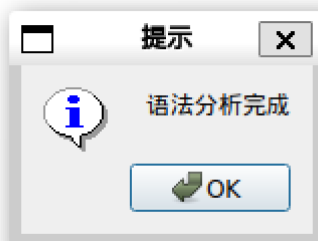
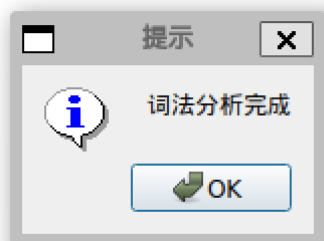


- 语法导入时间通常比词法导入时间短。
- 在成功导入后，会弹出窗口提示加载完成。



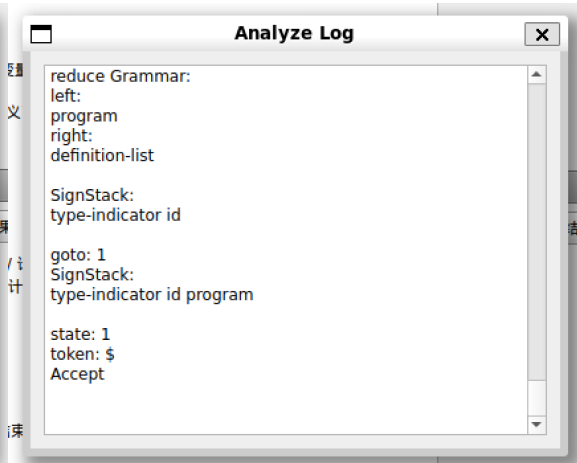
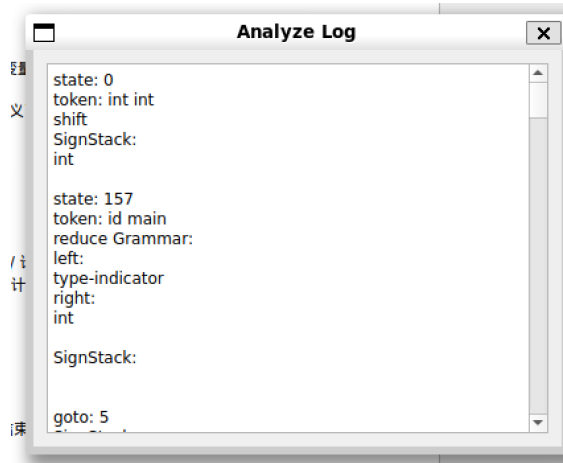
语法分析

- 点击按钮“语法分析”，即可对编辑框内的源程序进行语法分析。
- 会依次弹出词法分析是否成功和语法分析是否成功的窗口。



查看分析过程

- 在语法分析完成后，点击按钮“查看分析过程”，即可查看分析过程。



导入语义函数

- 语义函数的要求：
 - 在文件头部包含 `Include "<项目根目录> /grammarAnalyzer /analyzer /analyzer.h"` 以获得数据结构 `Tree` 的定义。
 - 在 `void fixTree(Tree *tree)` 内部进行实现。
- 编写语义函数的建议：
 - 参考以下模板：

```
namespace std{
extern "C" {
set<string> ignoreSigns = {"space", "tab", "enter", "(", ")",
";", "[", ""]};
set<string> ops = {"+", "-", "*", "/", ">", "<", "^", "%",
"<=", ">=", "=", "<>"};
void fixTree(Tree * tree){
// 递归处理树
if (tree->children.empty()){
return;
}
for (auto child : tree->children){
fixTree(child);
}
vector<Tree*> newChildren;
// 对特定符号进行处理
// exp （这里处理二元运算符，三元运算符需要参考下面if例子处理
```

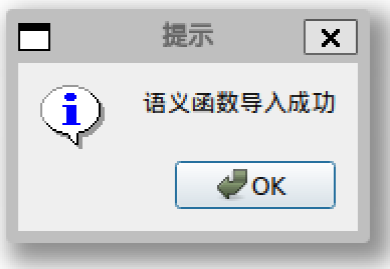
```

        if (tree->children.size() == 3 && ops.find(tree->children[1]->sign) != ops.end()){
            tree->value = tree->children[1]->value;
            tree->children[1]->value = "";
        }
        // if 参考文法结构进行处理，需要删除的节点只需将其value设为""即可
        if (tree->sign == "if-stmt"){
            tree->value = tree->children[0]->value;
            tree->children[0]->value = "";
            tree->children[2]->children = tree->children[3]->children;
            tree->children[3]->value = "";
            if (tree->children[4]->sign == "else"){
                tree->children[4]->children = tree->children[6]->children;
                tree->children[5]->value = "";
            }
        }
        // 删除应该被忽略的符号（定义在ignoreSigns中）
        for (auto child : tree->children){
            if (ignoreSigns.find(child->sign) == ignoreSigns.end()){
                newChildren.push_back(child);
            }
        }
        tree->children = newChildren;
        // 将只有一个子节点且value值为空的符号合并
        if (tree->children.size() == 1 && tree->value == ""){
            tree->sign = tree->children[0]->sign;
            tree->value = tree->children[0]->value;
            tree->children = tree->children[0]->children;
        }
        // 删除value为空的符号
        newChildren.clear();
        for (auto child : tree->children){
            if (!(child->value == "")){
                newChildren.push_back(child);
            }else{
                for (auto c : child->children){
                    newChildren.push_back(c);
                }
            }
        }
        tree->children = newChildren;
    }
}
}

```

导入语义函数

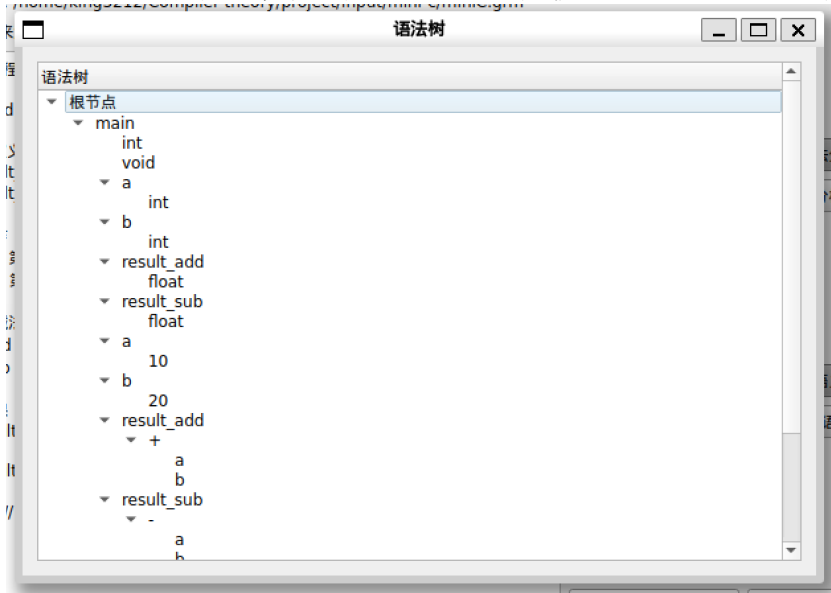
- 点击“导入语义函数”按钮，会打开一个文件资源管理器用于选择已经编写好的语义函数文件。
- 若完成选择，待语义函数处理完成后就会弹出提示语义函数导入完成。



- 若语义函数存在问题，会提示导入失败，并附上编译命令，可在终端尝试，获取具体问题。

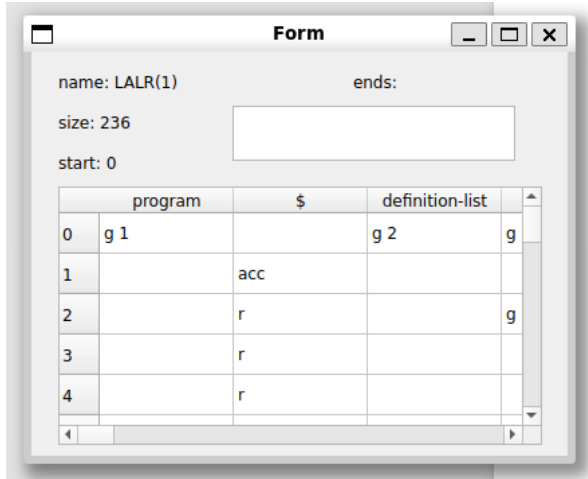
查看语法树

- 语法树导入成功后，点击“查看语法树”按钮即可查看语法树。



查看LALR1DFA

- 点击“查看LALR1DFA”按钮即可查看LALR1DFA边集合。



查看LR1DFA

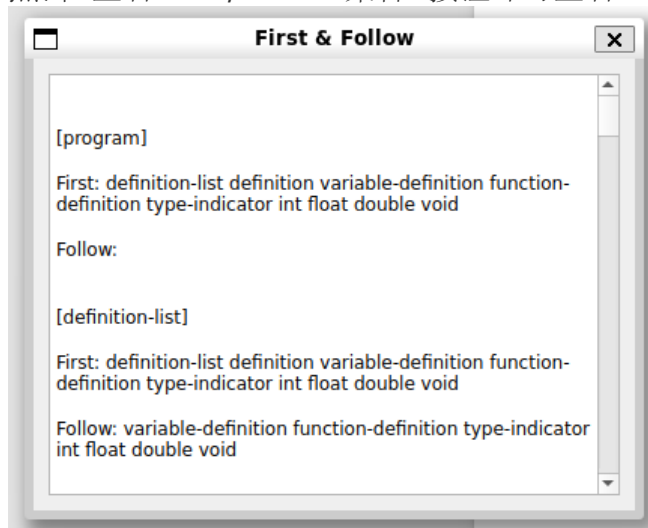
- 点击“查看LR1DFA”按钮即可查看LR1DFA边集合。
- 效果同上。

查看LALR(1)分析表

- 点击“查看LALR(1)分析表”按钮即可查看分析表。

查看First、Follow集合

- 点击“查看First/Follow集合”按钮即可查看First、Follow集合。



其他功能

如果需要清除预构建的内容，只需要执行：

```
make clean
```

之后重新编译运行即可